

实验报告

实验题目：基于深度伪造技术的人脸伪造实验

课程题目：多媒体安全技术

1、实验目的和要求

利用面部特征点检测、仿射变换、掩码计算与处理等技术，在 Python 环境下实现人脸特征点绘制，并完成人脸伪造操作。

2、实验设备（环境）

pycharm2022.2.2、python3.10.9、dlib-19.22.99、numpy-1.23.5、opencv-python 4.10.0.84。

3、实验步骤

- 首先安装环境，python 版本为 3.10.9，必要库：dlib、numpy、opencv

```
(1) PS E:\作业\计算机设计\pythonProject> python --version
Python 3.10.9
```

```
(1) PS E:\作业\计算机设计\pythonProject> pip list
WARNING: Ignoring invalid distribution -ip (e:\作业\多媒体\1\lib\site-packages)
Package           Version
-----
dlib               19.22.99
numpy              1.23.5
opencv-python     4.10.0.84
pip                25.1
setuptools         65.5.0
wheel              0.37.1
```

安装成功后运行 Test_version.py，测试实验环境是否安装成功，若无输出则成功安装。

- 考点 1：在原图上绘制特征点并保存图像，考点描述：请编写代码，绘制两张图像的特

征点并保存为图像文件，文件名分别为 image1_with_landmarks.jpg 和 image2_with_landmarks.jpg。

我们在 Test_version.py 的基础上添加函数 draw_landmarks(), 绘制特征点，由于原代码中已经存在提取特征点的函数 read_im_and_landmarks, 我们只需要将其绘制出来并保存即可。

我们遍历特征点，在原图像上进行绘制（cv2.circle 绘制圆形）后保存图像（cv2.imwrite）为 image1_with_landmarks.jpg 和 image2_with_landmarks.jpg。

```
def draw_landmarks(im, landmarks):  
  
    im = im.copy()  
  
    for point in landmarks:  
  
        center = (point[0, 0], point[0, 1])  
  
        cv2.circle(im, center, radius=2, color=(0, 255, 0), thickness=-1)  
  
    return im  
  
im1_with_landmarks = draw_landmarks(im1.copy(), landmarks1)  
im2_with_landmarks = draw_landmarks(im2.copy(), landmarks2)  
  
cv2.imwrite('image1_with_landmarks.jpg', im1_with_landmarks)  
  
cv2.imwrite('image2_with_landmarks.jpg', im2_with_landmarks)
```

- 考点 2: 补全仿射变换矩阵,请查看 FaceSwap.py 文件, transformation_from_points 函数以提供, 了解平移和缩放操作。

补全点 1: 利用 np.mean()函数,axis=0 表示计算每一列的均值。

c1、c2 分别表示两组点的平均坐标.

```
c1 = np.mean(points1, axis=0)
```

```
c2 = np.mean(points2, axis=0)
```

补全点 2: 利用 `np.std()` 函数得到两个特征点的标准差, 标准差是用来衡量一组数据离散程度的统计量。对于一组数据而言, 标准差越大, 数据就越分散; 标准差越小, 数据就越集中。因此计算得到的 `s1` 和 `s2` 代表了他的数据尺度, 我们进行缩放就是把两个数组中的数据缩放到了一个相同的尺度中 (这里我们使用 `points /= s`, 把 `points` 中的每个元素都除以对应 `s`, 就可以让 `points` 数组中的数据分布在以 0 为中心, 标准差为 1 的尺度上)。

```
s1 = np.std(points1)
s2 = np.std(points2)
points1 /= s1
points2 /= s2
```

使用奇异值分解 (SVD) 对中心化和缩放后的特征点进行处理, 得到旋转矩阵 `R`, 使两组点的线性变换误差最小。

最终的仿射变换矩阵包含缩放、平移和旋转三部分。

- 考点 3: 补全 `get_face_mask` 函数, 已提供函数模板 `get_face_mask()`, 只需要完成掩码计算即可, 后续对掩码进行模糊处理, 得到平滑的过渡效果已经提供。

补全点 1: 步骤一: 创建空掩码, 利用 `np.zeros()` 函数, 作为存储人脸掩码的基础。

```
mask = np.zeros(im.shape[:2], dtype=np.float64)
```

补全点 2: 利用 `cv2.fillConvexPoly()` 函数, 参数一为步骤一创建的空掩码 (`mask`), 参数二为凸包坐标点 (`hull`), 参数三为: `color=1`, 用于填充凸包, 填充颜色为 1。在掩码数组中将 `hull` (人脸大致轮廓范围) 进行填充, 明确人脸所在区域。

```
cv2.fillConvexPoly(mask, hull, color=1)
```

```
mask = np.array([mask, mask, mask]).transpose((1, 2, 0))

mask = (cv2.GaussianBlur(mask, (FEATHER_AMOUNT, FEATHER_AMOUNT),
0) > 0) * 1.0

mask = cv2.GaussianBlur(mask, (FEATHER_AMOUNT, FEATHER_AMOUNT),
0)

return mask
```

后续利用除法结合掩码（注意是掩码，源代码是对 im 进行操作，我们这里需要修改为 mask）处理后的图像做高斯模糊，得到平滑的过渡效果。

- 最后修改完以上函数之后，我们在代码中加入 draw_landmarks 函数（考点 1），然后将相关代码注释取消：

```
im1_with_landmarks = draw_landmarks(im1.copy(), landmarks1)

im2_with_landmarks = draw_landmarks(im2.copy(), landmarks2)

cv2.imwrite('image1_with_landmarks.jpg', im1_with_landmarks)

cv2.imwrite('image2_with_landmarks.jpg', im2_with_landmarks)
```

以及

```
mask = get_face_mask(im2, landmarks2)
```

完成相关函数的引用，然后运行该脚本即可。

4、实验结果

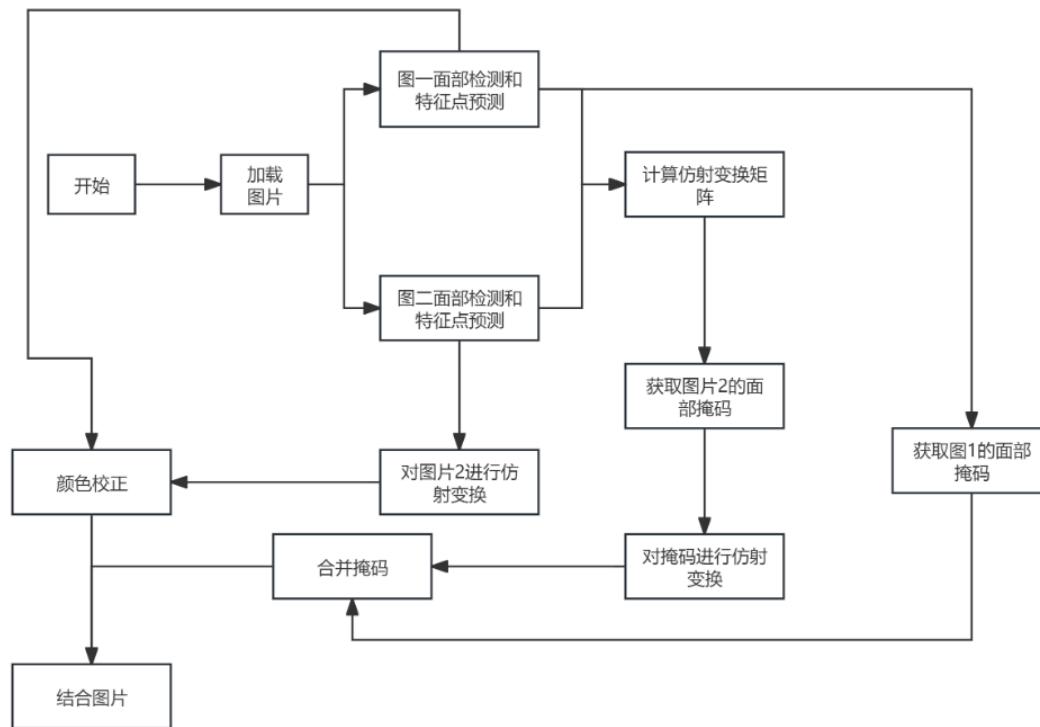
绘制特征点



进行人脸伪造



5、讨论和分析（分析实验原理、实验结果以及在实验中应注意的问题）



实验原理：

先检测两张图片面部特征点，计算仿射变换矩阵将其中一张图（图 2）进行仿射变换使其面部姿态等与另一张图（图 1）匹配；获取图 2 面部掩码并同样进行仿射变换，合并掩码后经颜色校正等处理，将变换后的图 2 面部区域融合到图 1，实现人脸伪造。

具体步骤：

- 面部特征点检测与绘制：利用 `dlib` 库等工具检测图像中面部特征点。`draw_landmarks` 函数通过遍历特征点坐标，使用 `cv2.circle` 在图像上以圆形标记出特征点位置，直观展示面部关键位置。
- 仿射变换矩阵计算：
 - 计算平均坐标：使用 `np.mean` 函数按列计算两组面部特征点的平均坐标 $c1$ 和 $c2$ ，这一步是为了后续将特征点进行中心化，去除位置差异影响。
 - 缩放特征点：借助 `np.std` 函数获取两组特征点的标准差 $s1$ 和 s ，通过将特征点除以各自标准差，使两组特征点在尺度上归一化，以相同尺度为后续处理做准备。
 - 奇异值分解(SVD)：对中心化和缩放后的特征点进行 SVD 操作得到旋转矩阵 R ，结合前面的缩放、平移信息构建仿射变换矩阵，实现从一组特征点到另一组特征点的线性变换，保证变换误差最小。
- 面部掩码获取：
 - 创建空掩码：用 `np.zeros` 根据图像尺寸创建二维空掩码，为标记人脸区域提供基础结构。

- 填充凸包: 利用面部特征点计算凸包(通过 `cv2.convexHull()`), 再用 `cv2.fillConvexPoly` 在掩码中填充凸包区域, 确定人脸大致范围。
- 维度转换与模糊处理: 将二维掩码扩展为三维与彩色图像维度匹配, 然后通过高斯模糊 (`cv2.GaussianBlur()`) 使掩码边缘平滑过渡, 便于后续与图像融合操作。

实验结果:

成功在两张图上绘制出面部特征点, 直观呈现面部关键位置, 可用于检查特征点检测准确性, 若特征点位置偏差大, 会影响后续仿射变换和人脸融合效果。

人脸伪造图像从视觉上看面部姿态、表情过渡自然, 色彩融合和谐, 说明仿射变换、掩码处理及颜色校正等操作较成功。

实验问题:

环境配置: 确保 python 3.10.9 以及 dlib、numpy、opencv - python 等库版本准确安装且相互兼容, 版本不匹配可能导致函数调用出错、运行报错等问题。可通过 `pip list` 检查已安装库版本, 必要时重新安装或升级。

特征点检测准确性: 面部特征点检测受图像质影响大。低分辨率或光照不均图像可能使特征点检测不准确, 导致后续仿射变换和融合出现偏差。

掩码处理: 创建掩码时确保尺寸与图像匹配, 填充凸包操作要正确获取凸包坐标, 否则掩码不能准确标记人脸区域。维度转换后进行高斯模糊, 模糊核大小设置要合适, 过大可能使面部区域过度模糊丢失细节, 过小则融合过渡不自然。